

C++ Complete Revision Guide

Libraries, Syntax, STL, Constants & Everything for Coding

TCS NQT Preparation Material
Generated on: April 07, 2026

1. Essential C++ Headers / Libraries

Header files you need to include for various functionalities.

1.1 The Universal Header

For competitive programming, one header includes everything:

```
#include <bits/stdc++.h>
using namespace std;
```

Tip: This includes ALL standard libraries. Use it for contests/practice. For production code, include specific headers.

1.2 Specific Headers (Production Style)

Header	Purpose	Example Use
<iostream>	Input/Output streams	cin, cout, cerr, endl
<vector>	Dynamic arrays	vector<int>
<string>	String class	string s = "hello"
<algorithm>	Sort, search, reverse	sort(), reverse(), find()
<map>	Ordered map (RB tree)	map<int,int>
<unordered_map>	Hash map	unordered_map<int,int>
<set>	Ordered set	set<int>
<unordered_set>	Hash set	unordered_set<int>
<queue>	Queue & priority queue	queue<int>, priority_queue<int>
<stack>	Stack	stack<int>
<deque>	Double-ended queue	deque<int>
<list>	Doubly linked list	list<int>
<utility>	pair, swap, move	pair<int,int>
<tuple>	Tuple type	tuple<int,int,int>
<climits>	INT_MAX, INT_MIN, etc.	INT_MAX, LLONG_MAX
<cmath>	Math functions	sqrt, pow, abs, log
<cstdlib>	atoi, malloc, rand	atoi(), rand()
<cstring>	C string functions	strlen, strcpy, memset
<numeric>	accumulate, gcd, lcm	accumulate(), __gcd()
<bitset>	Fixed-size bit array	bitset<32>
<sstream>	String streams	stringstream
<iomanip>	I/O formatting	setprecision, setw
<functional>	Function objects	greater<int>, less<int>

<chrono>	Time measurements	chrono::steady_clock
----------	-------------------	----------------------

2. Basic Program Structure

2.1 Minimal Template

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Your code here
    return 0;
}
```

2.2 Fast I/O Template (Recommended)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];

    // Solve

    return 0;
}
```

Tip: `sync_with_stdio(false)` makes `cin/cout` 5-10x faster. Use this for large inputs.

2.3 Multiple Test Cases Template

```
int main() {
    int t;
    cin >> t;
    while (t--) {
        // Solve each test case
    }
    return 0;
}
```

3. Data Types & Sizes

3.1 Primitive Types

Type	Size (bytes)	Range	Format Specifier
bool	1	true / false	%d
char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
short	2	-32,768 to 32,767	%hd
int	4	$-2.1 \cdot 10^9$ to $2.1 \cdot 10^9$	%d
unsigned int	4	0 to $4.2 \cdot 10^9$	%u
long	4 or 8	Platform dependent	%ld
long long	8	$-9.2 \cdot 10^{18}$ to $9.2 \cdot 10^{18}$	%lld
unsigned long long	8	0 to $1.8 \cdot 10^{19}$	%llu
float	4	~6 decimal digits	%f
double	8	~15 decimal digits	%lf
long double	12 or 16	~18 decimal digits	%Lf

3.2 When to use which type

Tip: Use 'int' for normal integers (up to ~2 billion).

Tip: Use 'long long' when values exceed $2 \cdot 10^9$ (e.g., factorials, products).

Tip: Use 'double' for floating point (more precision than float).

Tip: Use 'long long' for sums of large arrays to avoid overflow.

4. Constants (climits, cfloat)

4.1 Integer Limits (#include)

Constant	Value	Description
CHAR_MIN	-128	Minimum char value
CHAR_MAX	127	Maximum char value
UCHAR_MAX	255	Max unsigned char
SHRT_MIN	-32768	Minimum short
SHRT_MAX	32767	Maximum short
USHRT_MAX	65535	Max unsigned short
INT_MIN	-2,147,483,648	Minimum int (-2^{31})
INT_MAX	2,147,483,647	Maximum int ($2^{31} - 1$)
UINT_MAX	4,294,967,295	Max unsigned int ($2^{32}-1$)
LONG_MIN	-2^{31} or -2^{63}	Minimum long
LONG_MAX	$2^{31}-1$ or $2^{63}-1$	Maximum long
LLONG_MIN	$-9.2 * 10^{18}$	Minimum long long (-2^{63})
LLONG_MAX	$9.2 * 10^{18}$	Maximum long long ($2^{63}-1$)
ULLONG_MAX	$1.8 * 10^{19}$	Max unsigned long long

4.2 Float Limits (#include)

Constant	Description
FLT_MAX	Maximum float ($\sim 3.4 * 10^{38}$)
FLT_MIN	Minimum positive float
DBL_MAX	Maximum double ($\sim 1.7 * 10^{308}$)
DBL_MIN	Minimum positive double
FLT_EPSILON	Smallest float such that $1.0 + \text{EPSILON} \neq 1.0$
DBL_EPSILON	Same for double ($\sim 2.2 * 10^{-16}$)

4.3 Custom Infinity Constants (Common Practice)

```
const int INF = 1e9;           //  $\sim 2 * 10^9$  (fits in int)
const long long LLINF = 1e18;  // For long long
const int MOD = 1e9 + 7;      // Common prime modulus
const double EPS = 1e-9;      // For floating point comparison
const double PI = 3.14159265358979323846;
```

Tip: Don't use INT_MAX as INF when adding values - it will overflow! Use 1e9 instead.

5. Input / Output

5.1 Basic cin / cout

```
int n;  
cin >> n;           // Read integer  
cout << n << endl;  // Print with newline  
cout << n << "\n";  // Faster than endl  
  
string s;  
cin >> s;           // Reads single word (no spaces)  
getline(cin, s);    // Reads entire line  
  
// Reading multiple values  
int a, b, c;  
cin >> a >> b >> c;
```

5.2 Reading Until EOF

```
int x;  
while (cin >> x) {  
    // Process x  
}
```

5.3 Reading a Line After Integer

```
int n;  
cin >> n;  
cin.ignore(); // Skip newline left in buffer  
string line;  
getline(cin, line);
```

5.4 Output Formatting

```
#include <iomanip>  
  
cout << fixed << setprecision(2) << 3.14159; // 3.14  
cout << setw(10) << 42;                       // Right-aligned width 10  
cout << setfill('0') << setw(5) << 42;         // 00042  
cout << hex << 255 << endl;                     // ff  
cout << oct << 8 << endl;                       // 10  
cout << dec << 10 << endl;                     // back to decimal
```

6. STL Containers

6.1 vector - Dynamic Array

```
#include <vector>

vector<int> v; // Empty vector
vector<int> v(10); // Size 10, all zeros
vector<int> v(10, 5); // Size 10, all 5s
vector<int> v = {1, 2, 3}; // Initializer list
vector<vector<int>> mat(n, vector<int>(m, 0)); // 2D vector n*m

// Operations (Time Complexity)
v.push_back(x); // O(1) amortized
v.pop_back(); // O(1)
v[i]; // O(1)
v.at(i); // O(1) with bounds check
v.size(); // O(1)
v.empty(); // O(1)
v.clear(); // O(n)
v.front(); // O(1) - first element
v.back(); // O(1) - last element
v.insert(v.begin()+i, x); // O(n)
v.erase(v.begin()+i); // O(n)
v.resize(newSize); // O(n)
v.begin(), v.end(); // Iterators
```

6.2 string

```
#include <string>

string s = "hello";
s.length(); s.size(); // O(1)
s[i]; // O(1)
s.substr(start, len); // O(len)
s.find("sub"); // O(n*m), returns string::npos if not found
s.find('c'); // Find character
s += "world"; // O(m) amortized
s.append("abc"); // Same as +=
s.insert(pos, "abc"); // O(n)
s.erase(pos, len); // O(n)
s.replace(pos, len, "abc"); // O(n)
s.compare(s2); // 0 if equal

// Conversions
int x = stoi("42"); // string to int
long long y = stoll("42"); // string to long long
double d = stod("3.14"); // string to double
string s = to_string(42); // any number to string
```

6.3 pair & tuple

```
#include <utility>

pair<int, string> p = {1, "abc"};
p.first;           // 1
p.second;          // "abc"
make_pair(1, "abc");

// Tuple (3+ elements)
#include <tuple>
tuple<int, string, double> t = {1, "abc", 3.14};
get<0>(t);          // 1
get<1>(t);          // "abc"
auto [a, b, c] = t; // C++17 structured binding
```

6.4 map & unordered_map

```
#include <map>
#include <unordered_map>

// map - ordered (Red-Black Tree), O(log n) operations
map<string, int> m;
m["key"] = 10;           // Insert/update O(log n)
m.count("key");           // 0 or 1
m.find("key") != m.end(); // Check existence
m.erase("key");           // O(log n)
m.size();
for (auto& [k, v] : m) { ... } // Iterate (sorted by key)
m.lower_bound("abc");        // First key >= "abc"
m.upper_bound("abc");        // First key > "abc"

// unordered_map - hash table, O(1) average
unordered_map<string, int> um;
// Same operations as map but O(1) average, no ordering
```

Tip: Use unordered_map by default for speed. Use map when you need keys sorted.

6.5 set & unordered_set

```
#include <set>
#include <unordered_set>

set<int> s;           // Ordered, unique, O(log n)
s.insert(5);          // O(log n)
s.erase(5);           // O(log n)
s.count(5);            // 0 or 1
s.find(5);             // Returns iterator or s.end()
s.size(); s.empty();
*s.begin();           // Smallest element
*s.rbegin();          // Largest element
s.lower_bound(5);      // First >= 5
s.upper_bound(5);      // First > 5

// multiset - allows duplicates
multiset<int> ms;

// unordered_set - hash, O(1) average
unordered_set<int> us;
```

6.6 stack

```
#include <stack>

stack<int> st;
st.push(x);           // O(1)
st.top();              // O(1) - peek
st.pop();              // O(1) - removes top
st.size(); st.empty();
```

6.7 queue & deque

```
#include <queue>

queue<int> q;                // FIFO
q.push(x);                  // O(1) - add to back
q.front();                   // O(1) - peek front
q.back();                    // O(1) - peek back
q.pop();                     // O(1) - removes front

// deque - double-ended queue
#include <deque>
deque<int> dq;
dq.push_front(x);            // O(1)
dq.push_back(x);             // O(1)
dq.pop_front();              // O(1)
dq.pop_back();               // O(1)
dq[i];                       // O(1) random access
```

6.8 priority_queue (Heap)

```
#include <queue>

// Max-heap (default)
priority_queue<int> maxHeap;
maxHeap.push(x);           // O(log n)
maxHeap.top();             // O(1) - largest
maxHeap.pop();             // O(log n)

// Min-heap
priority_queue<int, vector<int>, greater<int>> minHeap;

// Custom comparator (max-heap by second element of pair)
auto cmp = [](pair<int,int> a, pair<int,int> b) {
    return a.second < b.second; // < for max-heap
};
priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);
```

6.9 list & forward_list

```
#include <list>
list<int> lst;           // Doubly linked list
lst.push_back(x); lst.push_front(x);
lst.pop_back(); lst.pop_front();
lst.insert(it, x);       // O(1) given iterator
lst.erase(it);           // O(1) given iterator
lst.sort();              // O(n log n)
lst.reverse();           // O(n)
lst.unique();            // Remove consecutive duplicates
```

6.10 bitset

```
#include <bitset>
bitset<32> bs;           // 32 bits, all zero
bitset<8> b("10110010"); // From string
bitset<8> b2(42);        // From integer
bs.set(i);               // Set bit i to 1
bs.reset(i);             // Set bit i to 0
bs.flip(i);              // Toggle bit i
bs.count();              // Number of 1 bits
bs.test(i);              // Check if bit i is set
bs.to_string();          // Convert to string
bs.to_ulong();           // Convert to unsigned long
```

7. STL Algorithms (#include)

7.1 Sorting

```
vector<int> v = {3, 1, 4, 1, 5};

sort(v.begin(), v.end());           // Ascending
sort(v.begin(), v.end(), greater<int>()); // Descending

// Custom comparator (lambda)
sort(v.begin(), v.end(), [](int a, int b) {
    return a > b; // Descending
});

// Sort by absolute value
sort(v.begin(), v.end(), [](int a, int b) {
    return abs(a) < abs(b);
});

stable_sort(v.begin(), v.end()); // Maintains order of equal elements
partial_sort(v.begin(), v.begin()+k, v.end()); // First k smallest sorted
nth_element(v.begin(), v.begin()+k, v.end()); // kth element in place
```

7.2 Searching

```
// Linear search
auto it = find(v.begin(), v.end(), target);
if (it != v.end()) { /* found */ }

// Binary search (array MUST be sorted)
binary_search(v.begin(), v.end(), target); // returns bool
auto lb = lower_bound(v.begin(), v.end(), x); // first >= x
auto ub = upper_bound(v.begin(), v.end(), x); // first > x

// Convert iterator to index
int idx = lb - v.begin();

// Count occurrences in sorted array
int cnt = upper_bound(...) - lower_bound(...);
```

7.3 Min / Max

```
min(a, b);           // Min of two
max(a, b);           // Max of two
min({a, b, c, d});   // Min of multiple (C++11)
max({a, b, c, d});   // Max of multiple

// Min/max element in a range
auto it = min_element(v.begin(), v.end());
auto it = max_element(v.begin(), v.end());
int idx = min_element(v.begin(), v.end()) - v.begin();
```

```
// Both at once
auto [mn, mx] = minmax_element(v.begin(), v.end());
```

7.4 Modifying Operations

```
reverse(v.begin(), v.end());           // Reverse in place
rotate(v.begin(), v.begin()+k, v.end()); // Rotate left by k
fill(v.begin(), v.end(), 0);           // Fill with value
iota(v.begin(), v.end(), 1);           // Fill with 1, 2, 3, ...
```

```
// Remove duplicates (sorted array)
v.erase(unique(v.begin(), v.end()), v.end());
```

```
// Remove all occurrences of value
v.erase(remove(v.begin(), v.end(), val), v.end());
```

```
// Permutations
next_permutation(v.begin(), v.end());
prev_permutation(v.begin(), v.end());
```

```
// Random shuffle (C++17+)
random_shuffle(v.begin(), v.end());
```

7.5 Numeric Operations (#include)

```
// Sum of elements
int sum = accumulate(v.begin(), v.end(), 0);
long long sum = accumulate(v.begin(), v.end(), 0LL); // Use 0LL for long long

// Product
int prod = accumulate(v.begin(), v.end(), 1, multiplies<int>());

// Prefix sum
vector<int> prefix(v.size());
partial_sum(v.begin(), v.end(), prefix.begin());

// Adjacent differences
vector<int> diff(v.size());
adjacent_difference(v.begin(), v.end(), diff.begin());

// GCD and LCM (C++17)
int g = gcd(12, 18);           // 6
int l = lcm(12, 18);           // 36
int g = __gcd(12, 18);         // Built-in (also works pre-C++17)
```

7.6 Counting & Predicates

```
count(v.begin(), v.end(), x);           // Count occurrences of x
count_if(v.begin(), v.end(), [](int n) {
    return n > 0;
});

all_of(v.begin(), v.end(), [](int n) { return n > 0; });
any_of(v.begin(), v.end(), [](int n) { return n > 0; });
none_of(v.begin(), v.end(), [](int n) { return n > 0; });

// Find with predicate
auto it = find_if(v.begin(), v.end(), [](int n) {
    return n % 2 == 0;
});
```

7.7 Set Operations (Sorted Ranges)

```
vector<int> a = {1, 2, 3, 4};
vector<int> b = {3, 4, 5, 6};
vector<int> result;

set_union(a.begin(), a.end(), b.begin(), b.end(),
          back_inserter(result)); // {1,2,3,4,5,6}

set_intersection(a.begin(), a.end(), b.begin(), b.end(),
                 back_inserter(result)); // {3,4}

set_difference(a.begin(), a.end(), b.begin(), b.end(),
               back_inserter(result)); // {1,2}
```

8. Math Functions (#include <math.h>)

```
abs(-5);                // 5 (works for int, long, double)
fabs(-3.14);             // 3.14 (for floating point)

sqrt(16);                // 4.0
cbrt(27);                // 3.0
pow(2, 10);              // 1024.0
pow(2.5, 3);             // 15.625

exp(1);                  // e (2.718...)
log(e);                  // 1 (natural log)
log2(8);                 // 3
log10(100);              // 2

ceil(3.2);               // 4.0
floor(3.8);              // 3.0
round(3.5);              // 4.0
trunc(3.7);              // 3.0

sin(x); cos(x); tan(x);
asin(x); acos(x); atan(x);
atan2(y, x);             // Angle in correct quadrant

fmod(10.5, 3);           // 1.5 (floating point modulo)
hypot(3, 4);             // 5 (sqrt(3^2 + 4^2))
```

8.1 Integer Math Tricks

```
// Avoid using sqrt for integer square root (loss of precision)
int isqrt = (int)sqrt((double)n + 0.5);

// Integer power (manual)
long long power(long long base, long long exp) {
    long long res = 1;
    while (exp > 0) {
        if (exp & 1) res *= base;
        base *= base;
        exp >>= 1;
    }
    return res;
}

// Modular exponentiation
long long powMod(long long b, long long e, long long m) {
    long long res = 1; b %= m;
    while (e > 0) {
        if (e & 1) res = res * b % m;
        b = b * b % m;
        e >>= 1;
    }
    return res;
}

// Ceiling division
```

```
int ceilDiv = (a + b - 1) / b;
```

9. Bit Manipulation

9.1 Basic Operators

Operator	Name	Example
&	AND	$5 \& 3 = 1$
	OR	$5 3 = 7$
^	XOR	$5 \wedge 3 = 6$
~	NOT	$\sim 5 = -6$
<<	Left shift	$5 \ll 1 = 10$ (multiply by 2)
>>	Right shift	$5 \gg 1 = 2$ (divide by 2)

9.2 Common Bit Tricks

```
// Check if bit i is set
bool isSet = (n >> i) & 1;
bool isSet = n & (1 << i);

// Set bit i
n |= (1 << i);

// Clear bit i
n &= ~(1 << i);

// Toggle bit i
n ^= (1 << i);

// Check if power of 2
bool isPow2 = (n > 0) && ((n & (n - 1)) == 0);

// Lowest set bit
int lsb = n & (-n);

// Clear lowest set bit
n = n & (n - 1);

// Count of set bits
__builtin_popcount(n);           // for int
__builtin_popcountll(n);         // for long long

// Number of leading zeros
__builtin_clz(n);                 // int
__builtin_clzll(n);              // long long

// Number of trailing zeros
__builtin_ctz(n);

// Parity (1 if odd number of bits, else 0)
__builtin_parity(n);
```

```
// XOR swap (no temp variable)
a ^= b; b ^= a; a ^= b;
```

9.3 Iterating All Subsets of a Set

```
// Iterate all subsets of n elements
for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            // i-th element is in subset
        }
    }
}
```

10. String Operations - Detailed

```
string s = "Hello World";

// Length
int len = s.length();           // 11
int len = s.size();             // same

// Access
char c = s[0];                  // 'H'
s[0] = 'h';                     // Modify

// Substring
string sub = s.substr(6);        // "World" (from index 6 to end)
string sub = s.substr(6, 3);     // "Wor" (3 chars from index 6)

// Find
int pos = s.find("World");       // 6 (or string::npos if not found)
if (s.find("xyz") == string::npos) { /* not found */ }

// Replace
s.replace(6, 5, "There");        // "Hello There"

// Insert
s.insert(5, " beautiful");

// Erase
s.erase(5, 6);                  // Remove 6 chars from index 5

// Reverse
reverse(s.begin(), s.end());

// Convert case
transform(s.begin(), s.end(), s.begin(), ::tolower);
transform(s.begin(), s.end(), s.begin(), ::toupper);

// Character checks (#include <cctype>)
isalpha(c); isdigit(c); isalnum(c);
isupper(c); islower(c); isspace(c);
tolower(c); toupper(c);

// Split string by space
stringstream ss(s);
string word;
vector<string> words;
while (ss >> word) words.push_back(word);
```

11. Iterators

```
vector<int> v = {1, 2, 3, 4, 5};

// Forward iteration
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}

// Reverse iteration
for (auto it = v.rbegin(); it != v.rend(); ++it) {
    cout << *it << " ";
}

// Range-based for loop (C++11+) - PREFERRED
for (int x : v) cout << x << " ";
for (auto& x : v) x *= 2;           // By reference to modify
for (const auto& x : v) ...;       // Read-only, no copy

// With index
for (int i = 0; i < (int)v.size(); i++) {
    cout << i << ": " << v[i] << endl;
}

// Iterator arithmetic (random access only)
auto it = v.begin() + 2;           // Points to v[2]
int idx = it - v.begin();          // 2
```

12. Lambda Functions (C++11+)

```
// Basic lambda
auto add = [](int a, int b) { return a + b; };
int x = add(3, 4);           // 7

// Capture by value
int multiplier = 10;
auto times = [multiplier](int x) { return x * multiplier; };

// Capture by reference
int counter = 0;
auto inc = [&counter]() { counter++; };

// Capture all by value / reference
auto f1 = [=]() { /* all variables by value */ };
auto f2 = [&]() { /* all variables by reference */ };

// Lambda as comparator
sort(v.begin(), v.end(), [](int a, int b) {
    return a > b; // descending
});

// Recursive lambda (C++14+)
function<int(int)> fact = [&](int n) {
    return n <= 1 ? 1 : n * fact(n - 1);
};
```

13. Classes and Structs

13.1 Basic Class

```
class Point {
private:
    int x, y;
public:
    // Constructor
    Point() : x(0), y(0) {}
    Point(int a, int b) : x(a), y(b) {}

    // Member functions
    int getX() const { return x; }
    int getY() const { return y; }
    void setX(int a) { x = a; }

    // Destructor
    ~Point() {}
};

Point p(3, 4);
Point* pp = new Point(5, 6);
delete pp;
```

13.2 Struct (for DSA)

```
// Linked List Node
struct ListNode {
    int val;
    ListNode* next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode* n) : val(x), next(n) {}
};

// Tree Node
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

14. Pointers and References

```
int a = 10;
int* p = &a;           // Pointer to a
cout << *p;            // 10 (dereference)
*p = 20;               // a is now 20

// Null pointer
int* p = nullptr;      // C++11+ (preferred)
int* p = NULL;         // C-style

// Reference (alias)
int& ref = a;           // ref is another name for a
ref = 30;              // a is now 30

// Pass by reference (avoid copy)
void modify(vector<int>& v) { // No copy, can modify
    v.push_back(1);
}

// Pass by const reference (no copy, read-only)
void print(const vector<int>& v) {
    for (int x : v) cout << x;
}

// Dynamic memory
int* arr = new int[10]; // Allocate array of 10 ints
delete[] arr;          // Free array

int* x = new int(5);    // Single int initialized to 5
delete x;

// Smart pointers (C++11+) - prefer these
#include <memory>
unique_ptr<int> up = make_unique<int>(10); // C++14
shared_ptr<int> sp = make_shared<int>(10);
```

15. Common Coding Patterns

15.1 Two Pointers

```
int left = 0, right = n - 1;
while (left < right) {
    if (condition) left++;
    else right--;
}
```

15.2 Sliding Window

```
int left = 0, sum = 0, ans = 0;
for (int right = 0; right < n; right++) {
    sum += arr[right];
    while (sum > target) {
        sum -= arr[left++];
    }
    ans = max(ans, right - left + 1);
}
```

15.3 Binary Search

```
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2; // Avoid overflow
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
return -1;
```

15.4 BFS Template

```
queue<int> q;
vector<bool> visited(n, false);
q.push(start);
visited[start] = true;
while (!q.empty()) {
    int node = q.front(); q.pop();
    for (int next : adj[node]) {
        if (!visited[next]) {
            visited[next] = true;
            q.push(next);
        }
    }
}
```

15.5 DFS Template (Recursive)

```
void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true;
    for (int next : adj[node]) {
        if (!visited[next]) {
            dfs(next, adj, visited);
        }
    }
}
```

15.6 Prefix Sum

```
vector<int> prefix(n + 1, 0);
for (int i = 0; i < n; i++)
    prefix[i + 1] = prefix[i] + arr[i];
// Sum of arr[l..r] = prefix[r+1] - prefix[l]
```

15.7 DP Template

```
// 1D DP
vector<int> dp(n + 1, 0);
dp[0] = base_case;
for (int i = 1; i <= n; i++)
    dp[i] = /* recurrence */;

// 2D DP
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
```

16. Time Complexity Cheat Sheet

16.1 Common Algorithms

Algorithm	Best	Average	Worst	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

16.2 STL Operations

Container	Access	Search	Insert	Delete
vector	$O(1)$	$O(n)$	$O(1)$ end	$O(1)$ end
deque	$O(1)$	$O(n)$	$O(1)$ ends	$O(1)$ ends
list	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
set/map	-	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_set/map	-	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
stack/queue	$O(1)$ end	-	$O(1)$	$O(1)$
priority_queue	$O(1)$ top	-	$O(\log n)$	$O(\log n)$

(* with iterator)

16.3 What's Acceptable for Different N

Input Size n	Acceptable Complexity
$n \leq 10$	$O(n!)$ - permutations OK
$n \leq 20$	$O(2^n)$ - subsets OK
$n \leq 500$	$O(n^3)$
$n \leq 5,000$	$O(n^2)$

$n \leq 10^6$	$O(n \log n)$
$n \leq 10^8$	$O(n)$ or $O(\log n)$
$n > 10^8$	$O(\log n)$ or $O(1)$

17. Common Pitfalls & Mistakes

1. Integer Overflow

```
// WRONG: int overflows for large n
int n = 1000000;
int result = n * n;           // OVERFLOW!
```

```
// CORRECT: use long long
long long result = (long long)n * n;
```

2. Array Index Out of Bounds

```
// WRONG: comparing signed/unsigned
for (int i = 0; i < v.size() - 1; i++) // If v is empty, size()-1 underflows!
```

```
// CORRECT
for (int i = 0; i + 1 < (int)v.size(); i++)
```

3. Modifying Container While Iterating

```
// WRONG
for (auto x : v) {
    if (x == 0) v.erase(...); // Invalidates iterator!
}
```

```
// CORRECT - use erase-remove idiom
v.erase(remove(v.begin(), v.end(), 0), v.end());
```

4. Uninitialized Variables

```
int x; // Garbage value!
int x = 0; // OK
vector<int> v(10); // All zeros (vector initializes)
int arr[10]; // Garbage values
int arr[10] = {0}; // All zeros
```

5. Comparing Floating Point

```
// WRONG
if (a == b) ... // May fail due to precision
```

```
// CORRECT
if (abs(a - b) < EPS) ...
```

6. Recursive Stack Overflow

Default stack size is ~1 MB. Deep recursion ($>10^5$) may overflow. Use iterative or increase stack.

7. Forgetting to Reset Global State

In multi-test-case problems, reset global vectors/arrays before each test case.

18. Compilation & Debugging

18.1 Compilation Commands

```
# Basic compilation
g++ solution.cpp -o solution

# Recommended (with optimizations and warnings)
g++ -std=c++17 -O2 -Wall -Wextra solution.cpp -o solution

# Debug build
g++ -std=c++17 -g -Wall solution.cpp -o solution_debug

# Run
./solution
./solution &lt; input.txt           # Input from file
./solution &lt; input.txt &gt; output.txt # I/O from files

# C++ standards
-std=c++11  # C++11 (auto, lambda, range-for)
-std=c++14  # Adds generic lambdas
-std=c++17  # Adds structured bindings, if-init
-std=c++20  # Adds concepts, ranges
```

18.2 Debug Macros

```
// Debug print macro
#define debug(x) cerr &lt;&lt; #x &lt;&lt; " = " &lt;&lt; x &lt;&lt; endl

// Use:
int x = 42;
debug(x);           // x = 42

// Print vector
for (int v : vec) cerr &lt;&lt; v &lt;&lt; " "; cerr &lt;&lt; endl;
```

19. Useful One-Liners & Snippets

```
// Sum of vector
long long sum = accumulate(v.begin(), v.end(), 0LL);

// Max in vector
int mx = *max_element(v.begin(), v.end());

// Min in vector
int mn = *min_element(v.begin(), v.end());

// Sort descending
sort(v.rbegin(), v.rend());

// Unique elements (sort first!)
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());

// Frequency map
unordered_map<int, int> freq;
for (int x : v) freq[x]++;

// Check if element exists
if (find(v.begin(), v.end(), x) != v.end()) ...

// Convert vector to set
set<int> s(v.begin(), v.end());

// Reverse string
reverse(s.begin(), s.end());

// String to char array
const char* cstr = s.c_str();

// 2D vector initialization
vector<vector<int>> mat(n, vector<int>(m, 0));

// Print all elements
for (int x : v) cout << x << " ";
cout << endl;

// Random number
#include <random>
random_device rd;
mt19937 gen(rd());
uniform_int_distribution<> dis(1, 100);
int r = dis(gen);
```

20. C++ Keywords Quick Reference

Keyword	Purpose
auto	Type deduction (C++11+)
const	Read-only variable
constexpr	Compile-time constant (C++11+)
static	Class-level / file-scope variable
virtual	Polymorphic function
override	Explicit override (C++11+)
final	Cannot be overridden / inherited
explicit	Prevent implicit conversion
friend	Grant access to private members
inline	Hint to inline function
mutable	Can be modified in const object
volatile	Don't optimize away
typename	Indicate type (templates)
template	Generic programming
nullptr	Null pointer (C++11+)
sizeof	Size of type/object
typedef	Type alias (use 'using' instead)
using	Type alias (preferred)
namespace	Group declarations
enum / enum class	Enumeration
new / delete	Dynamic memory
this	Pointer to current object
throw / try / catch	Exception handling

Type Aliases

```
// using is preferred over typedef
using ll = long long;
using vi = vector<int>;
using vvi = vector<vector<int>>;
using pii = pair<int, int>;
using vpai = vector<pair<int, int>>;
```